



Yggdrasil Platform — B2B Event Hub Architecture

Mimir Labs Technical Publication

Document:	ML-WP-014	Version:	1.0
Author:	Christopher Gaither	Date:	April 2026
Classification:	Public		

1 Yggdrasil Platform — B2B Event Hub Architecture

Mimir Labs Technical Publication Document Version: 1.0 **Date:** April 2026 **Classification:** Public

1.1 Executive Summary

The B2B Event Hub is the real-time observability and integration substrate of Yggdrasil ERP. It is the system through which significant business activity becomes visible — to user interfaces, to integration partners, to other Yggdrasil tenants, and to downstream analytical and archival infrastructure — at the moment it occurs.

Most enterprise software treats event emission as a peripheral feature: a webhook surface bolted onto an otherwise polling-based architecture, or a vendor-mediated event mesh sold as a separate SKU. Yggdrasil treats real-time event flow as a first-class architectural concern. The hub is built into the server, governed by the same authorization envelope that protects every other API surface, persisted to the same canonical database, and federated through pluggable relays for cross-node deployment.

This paper describes the hub's protocol, persistence model, authentication flow, delivery semantics, federation modes, retry guarantees, and operational characteristics.



1.2 1. Architectural Role

The hub serves four distinct operational responsibilities within the platform.

Live UI updates. Desktop clients and the web application receive state changes the moment they occur. There is no polling layer between user action and screen refresh. When a sales order moves from draft to confirmed in one operator's session, every other authenticated client viewing that tenant's order pipeline observes the transition immediately.

Integration substrate. Third-party systems subscribe to the same event stream the internal clients use. There is no separate webhook subsystem, no second-class integration API, and no per-event refetch tax. An integration partner authenticated against a tenant receives the same payload as the desktop client running for that tenant.

Cross-tenant publication. Yggdrasil is multi-tenant by design. The hub supports publication of events from one tenant to another with full audit retention, enabling B2B workflows — supplier-to-buyer, buyer-to-supplier, parent-to-subsidary — without requiring an external integration tier.

Federation backbone. Multi-node deployments use the hub's relay layer to maintain event coherence across servers. A change persisted on one node propagates to all other nodes through the configured federation transport, enabling horizontal scale without sacrificing the real-time delivery guarantee.

Each of these responsibilities is served by the same protocol, the same persistence model, and the same authorization envelope. The hub is one architectural component, not four.

1.3 2. Protocol

The hub exposes a WebSocket endpoint on the platform's configured B2B port (8081 by default). All hub interactions occur through structured JSON messages over this single connection.

1.3.1 Connection Lifecycle

A client establishes a WebSocket connection. The server accepts the connection in an unauthenticated state. The client must send an authentication message before any other message will be processed; messages sent prior to authentication receive an error response and the connection is held but inactive.

```
{ "type": "auth", "token": "Bearer <jwt>" }
```

The token is the same JWT that authenticates HTTP API requests. The hub validates it through the platform's `AuthManager` and extracts the tenant identifier, user identifier, and role claims. The connection's tenant context is bound to the validated token and persists for the lifetime of the connection.



If the token is invalid or expired, the server returns an `auth_error` message and closes the connection. Tokens that expire mid-session do not silently drop the connection — the client is expected to refresh and reconnect.

1.3.2 Message Envelope

All messages — both client-to-server and server-to-client — share a common envelope:

```
{ "type": "<event_type>", "data": <payload> }
```

The `type` field identifies the event kind. Event types follow a `<module>.<entity>.<verb>` convention: `sales.order.confirmed`, `quality.ncr.opened`, `purchasing.po.received`. The `data` field carries the event payload as a structured object.

There is no separate schema registry. Event types and their payload structures are defined in the platform's published API documentation and follow the same naming and shape conventions as the corresponding REST resources.

1.3.3 Connection Tracking

Every authenticated connection is recorded in the `websocket_connections` table at the time of authentication. The record captures tenant identifier, user identifier, connect timestamp, last-activity timestamp, and client metadata. This table provides the operational visibility for connection health monitoring, idle-session reaping, and security audit.

When a connection terminates — whether by client disconnect, server-side close, or network failure — the corresponding record is cleared.

1.4 3. Publication Modes

The hub provides three distinct publication operations, each with different delivery semantics and persistence guarantees.

1.4.1 Local Broadcast

`broadcastEvent(eventType, data)` delivers an event to every authenticated client currently connected, regardless of tenant. This mode is reserved for system-wide notifications — global maintenance announcements, version upgrade notices, and similar platform-level signals — and is rarely used in business code.

Local broadcasts are not persisted. They are best-effort delivery to currently connected clients; clients that connect later do not receive them.



1.4.2 Tenant-Scoped Delivery

`sendToTenant(tenantId, eventType, data)` delivers an event to all authenticated clients whose connection is bound to the specified tenant. The hub iterates the active connection list, filters by the connection's authenticated tenant context, and sends the message only to matching clients. Clients connected to other tenants do not observe the event.

Tenant-scoped delivery is the primary publication mode for live UI updates. It is fast, low-overhead, and bypasses persistence — making it appropriate for transient events such as cursor positions, presence indicators, or interactive document collaboration.

This mode does not federate to other nodes and does not persist. Tenants connected to a different node in a federated deployment will not observe these events.

1.4.3 Persisted Publication

`publishEvent(sourceTenantId, destTenantId, eventType, data)` is the durable publication path. The operation executes in four sequenced phases:

1. **Persist as pending.** A row is inserted into the `b2b_events` table with status `pending`, retry counter zero, and the full payload serialized as JSONB. The insert returns the assigned event identifier.
2. **Local delivery.** The event is delivered to all authenticated clients in the destination tenant currently connected to the local node, using the same iteration and tenant-filter logic as `sendToTenant`.
3. **Federated forwarding.** If a federation relay is configured (Redis or Redpanda), the event is published to the relay. The relay handles transport to other nodes; receiving nodes deliver to their local clients.
4. **Mark processed.** Once the relay accepts the event for forwarding, the database row is updated to status `processed` with a `processed_at` timestamp.

If any phase fails, the event remains in the `b2b_events` table in `pending` or `failed` status and is picked up by the retry timer.

This is the publication path that integration partners subscribe to and the path that survives node restarts and network partitions.

1.5 4. Persistence Model

The `b2b_events` table is the canonical record of every persisted event. Its schema captures the fields necessary to support delivery retry, audit, and federated replay:



Field	Purpose
event_id	UUID primary key, returned to publishers
event_type	Module-qualified event identifier
source_tenant_id	Originating tenant
destination_tenant_id	Target tenant (may equal source for intra-tenant events)
payload	JSONB structured event body
status	pending, processed, or failed
retry_count	Number of delivery attempts
created_at	Initial insert timestamp
processed_at	Successful delivery timestamp
error_message	Last failure reason, populated on transition to failed

The table is append-only from the publisher's perspective. The hub updates `status`, `processed_at`, `retry_count`, and `error_message` as the event progresses through its delivery lifecycle, but the payload and identifying fields are immutable.

Because events live in the canonical PostgreSQL database, they participate in the same backup, replication, and operational tooling as every other Yggdrasil entity. There is no separate event store to maintain, no second backup pipeline to coordinate, and no second source of truth to reconcile.

1.5.1 Retention

The default deployment retains processed events indefinitely. Operators may configure a retention policy that archives events older than a configured threshold to cold storage, but no events are deleted by the hub itself. The platform treats event history as audit-grade data.

1.6 5. Delivery Guarantees

The hub provides at-least-once delivery for persisted events with a bounded retry budget.

1.6.1 At-Least-Once

Every persisted event is delivered to subscribers at least once, provided the subscriber connects within the retry window. Events are not removed from `pending` or `failed` status until they are successfully delivered or until the retry budget is exhausted.

Subscribers must be idempotent. The same event may be delivered more than once if a network failure interrupts acknowledgment, if a federated relay redelivers a previously processed event, or



if the retry timer fires before a slow subscriber acknowledges receipt.

The platform's recommended pattern is for subscribers to track the last processed `event_id` per stream and to discard events whose identifiers fall behind that watermark.

1.6.2 Retry Budget

The retry timer fires every 30 seconds. On each fire, the hub selects up to 50 events with status `pending` or `failed` and retry count below 3, ordered by creation timestamp. Each selected event is re-delivered through the same four-phase sequence as the original publication.

Events that exceed the retry budget remain in the table with status `failed` and the last error captured in `error_message`. They are not automatically retried further; operator intervention is required to either replay them manually or accept the loss.

This bound prevents pathological retry storms during sustained subscriber outages while preserving full audit visibility into events that could not be delivered.

1.6.3 Ordering

Within a single tenant pair, events are delivered in creation-timestamp order, subject to the limitations of the retry queue (an event that initially failed and is retried later will arrive after subsequent successful events). Ordering across tenant pairs is not guaranteed.

Subscribers requiring strict ordering should consume the persisted event stream by querying `b2b_events` directly with appropriate watermark semantics, rather than relying on the WebSocket delivery sequence.

1.7 6. Federation

Single-node deployments operate with the persistence and delivery semantics described above and need no federation. Multi-node deployments — whether for horizontal scale, geographic distribution, or high availability — require event coherence across servers, which the federation layer provides.

Federation is configured at server startup through the `FederationMode` setting in `server.conf`. The hub supports two relay implementations.

1.7.1 Redis Relay

The Redis relay uses Redis pub/sub for cross-node event distribution. Each node subscribes to tenant-keyed channels; publications fan out through the Redis broker.

Redis is the appropriate choice for deployments where the federation transport runs adjacent to



the application servers and operational simplicity is the priority. Redis persistence is not used — the relay treats Redis as a transport, not a store. Durable persistence remains the responsibility of the local `b2b_events` table.

1.7.2 Redpanda Relay

The Redpanda relay uses Kafka-protocol topics on a central Redpanda broker for cross-node distribution. The relay produces to and consumes from tenant-keyed topics through the platform's sidecar tunnel.

Redpanda is the appropriate choice for deployments that already use Kafka-protocol infrastructure for analytics, archival, or cross-system integration. The same broker that fans events between Yggdrasil nodes can be consumed by external analytics pipelines without an additional integration layer.

1.7.3 Relay Selection

Both relays expose the same internal interface to the hub. The hub itself is relay-agnostic; the choice of Redis or Redpanda is operational, not architectural. Only one relay is active at a time. Echo suppression — preventing a node from re-receiving its own publications — is handled inside the relay layer, not by the hub.

1.8 7. Authorization

Every operation against the hub is governed by the platform's authentication and authorization envelope.

Connection. Establishing a WebSocket connection requires no authentication, but no operations are accepted on an unauthenticated connection. The connection serves as a transport channel; the authentication step binds it to a tenant context.

Publication. Publishing an event requires the publisher to hold a role with `b2b.publish` permission for the source tenant. Cross-tenant publication additionally requires that an explicit B2B relationship exist between source and destination tenants in the platform's tenant-relationship table.

Subscription. Subscribers receive only events scoped to their authenticated tenant context. The hub filters every outbound message against the connection's tenant identifier before transmission. There is no client-side filtering and no opt-in to other tenants' streams.

Audit. Connection establishment, authentication, and authorization decisions are logged through the platform's standard audit infrastructure. The `websocket_connections` table provides the connection-lifecycle record; authorization rejections appear in the platform audit trail alongside HTTP API authorization decisions.



1.9 8. Operational Characteristics

The hub is designed to operate within a small operational surface.

Resource consumption. A single Yggdrasil server holds active WebSocket connections in memory at a fixed per-connection overhead. Connection counts in the low thousands per node are the design target. Higher counts are supported through horizontal scaling and federation.

Backpressure. Slow subscribers do not block fast publishers. The hub writes to each socket's send queue and continues; subscribers that cannot keep up accumulate queue depth at the OS layer and may eventually be disconnected by the network stack. The persisted event record remains available for replay through the `b2b_events` table.

Failure modes. A failed local delivery does not affect federated forwarding, and a failed federation does not affect local delivery. Each phase of `publishEvent` is observable independently in the database state. The retry timer recovers from transient failures without operator intervention.

Restart behavior. Hub state is stateless apart from the connection table. On restart, all connections are dropped; clients reconnect and re-authenticate. Pending events in `b2b_events` are picked up by the retry timer on the next fire.

1.10 9. Integration Patterns

The hub supports several common integration patterns directly, without requiring an external middleware tier.

Real-time mirror. A subscriber connects, authenticates, and consumes events for a tenant. Each event reflects a state change in the source system; the subscriber maintains a local mirror of relevant tenant data. This is the pattern Bifrost uses to maintain alignment between Yggdrasil and external systems.

Event-driven workflow. A subscriber consumes events of a specific type, takes action based on the payload, and publishes a follow-up event. Multi-step workflows that span systems can be expressed as event chains without polling or batch reconciliation.

Audit replication. A subscriber consumes the full event stream for a tenant and replicates it to a separate audit store. Because the persisted event record is append-only and timestamped, the replicated store provides an independent audit trail suitable for compliance verification.

Cross-tenant B2B. A publisher in one tenant emits events targeted at a partner tenant. The destination tenant's subscribers receive the events through the same WebSocket protocol they use for their own tenant's events. No external EDI gateway, no shared filesystem, and no third-party integration platform is involved.



1.11 10. Comparison to External Approaches

The hub is functionally distinct from three categories of integration infrastructure that enterprise software typically relies on.

Webhooks. Webhooks are HTTP callbacks triggered by source-system events. They typically convey occurrence (“order updated”) without delta and require subscribers to refetch the affected resource to determine what changed. The hub conveys the full structured payload of the change in the original delivery and does not require a refetch step.

Event mesh products. Vendor-mediated event mesh products (SAP Event Mesh, Salesforce Platform Events) provide event distribution through vendor-hosted infrastructure with separate licensing. The hub is an integral component of the platform with no separate licensing or hosting concern.

Integration platforms. iPaaS products (Mulesoft, Boomi, Workato) provide hosted middleware that bridges systems through adapters and transformations. The hub does not replace iPaaS for organizations that need to bridge multiple non-Yggdrasil systems, but it removes the need for iPaaS as an intermediary between Yggdrasil and a subscriber that can speak WebSocket and consume structured JSON.

The hub does not aspire to be an event mesh, an iPaaS, or a workflow orchestrator. It is the event surface of one platform, designed to make that platform’s operational state continuously observable. For broader cross-system synchronization, Bifrost layers on top of the hub and provides the connector and routing logic for external system pairings.

1.12 Conclusion

The B2B Event Hub is the architectural choice that turns Yggdrasil from a transactional system of record into a continuously observable operational surface. Every consequential mutation is captured, persisted, and made available to authorized subscribers in real time, through one protocol, with one authorization envelope, and with one persistence model.

This design avoids the architectural fragmentation that characterizes enterprise software’s typical response to integration requirements: separate webhook subsystems, separate event mesh products, separate iPaaS layers, and separate audit pipelines, each with its own authorization model and operational concerns. The hub consolidates these concerns into a single component that is part of the platform rather than adjacent to it.

The result is a system whose operational state is continuously visible, whose integration cost is bounded by what subscribers do with the events rather than by what the platform charges per delivery, and whose audit posture is a structural property rather than a configurable feature.



Copyright 2026 Mimir Labs. All rights reserved.

Mimir Labs LLC | Harrisburg, PA

© 2024–2026 Mimir Labs LLC. All rights reserved.

This document contains proprietary information belonging to Mimir Labs LLC. No part of this publication may be reproduced, distributed, or transmitted in any form without the prior written permission of Mimir Labs LLC. The information herein is provided “as is” without warranty of any kind. Product names and trademarks referenced in this document are the property of their respective owners.